
Machine Unlearning against data poisoning attacks

Mathieu Charpentier, Alicia Shao

Polytechnique Montréal

Mat. 2403696, Mat. 2409849

Abstract

Machine learning models are increasingly deployed in critical applications relying on vast amounts of personal data. In response to growing privacy concerns, machine unlearning has emerged as a framework that aims to remove the influence of specific data from trained models. While most unlearning methods focus on user privacy, recent studies explore their role in defending against data poisoning attacks. In this work, we evaluate the Forsaken unlearning method, based on neuron masking, across a wide range of poisoning scenarios: label flipping, out-of-distribution injection, and Gaussian noise poisoning. We conduct extensive empirical evaluations on MNIST, including hyperparameter tuning and analysis, and use the Gaussian Unlearning Score (GUS) as a diagnostic tool. Our results suggest Forsaken is effective under careful tuning, though performance degradation must be mitigated.

1 Introduction

Nowadays, machine learning models are used in critical fields such as medical diagnosis, credit-risk assessment, surveillance, or advertising. Their training requires large amounts of personal data, which raises legitimate concerns about user privacy. With legislations like the European Union’s General Data Protection Regulation and the California Consumer Privacy Act and amid raising awareness about privacy, users are rightfully incentivized to revoke access to their data. However, removing user data from a trained model’s training database is not sufficient to guarantee their right to be forgotten. Overparameterized and overfitted machine learning models learn and remember their training data [Nguyen et al., 2024]. To this end, machine unlearning [Cao and Yang, 2015], an emerging field of research aims to fully remove a user’s data from machine learning models while maintaining existing accuracy [Nguyen et al., 2024].

The most reliable way to unlearn a subset of training data would be to retrain a new model from scratch [Ma et al., 2023] without the data that needs to be forgotten. This process, called **exact machine unlearning**, is costly, time and computation-wise, even more so with large-scale models [Pawelczyk et al., 2025]. Some data-driven approaches exist, based on data partitioning and efficient retraining [Bourtoule et al., 2019, Nguyen et al., 2024], but most proposed unlearning methods are **approximate**. Machine unlearning is then done by performing less computationally costly computations on the weights, by modifying the architecture or by filtering the outputs [Thudi et al., 2021, Nguyen et al., 2024].

Evaluation of such unlearning algorithms involves measuring model accuracy, misclassifications rates and comparing them to original performances [Pawelczyk et al., 2025]. Another method lies in verification through membership inference attacks [Pawelczyk et al., 2025, Nguyen et al., 2024], which predict if a specific data point was part of the training dataset, and other evaluation metrics. Thudi et al. [2021] use the L_2 distance between the weights of the model retrained from scratch and the unlearned weights, or verification error. Because it requires training of an exact unlearned model, Thudi et al. [2021] propose the unlearning error as a proxy that overcomes this drawback.

If the original motivation behind machine unlearning lies in guaranteeing user privacy, its application as a protection from backdoors and data poisoning attacks is also being investigated [Ma et al., 2023, Pawelczyk et al., 2025]. On the one hand, Pawelczyk et al. [2025] find that most popular approximate unlearning techniques for neural networks fail to prevent data poisoning attacks, the goal of which is to alter the behavior of a model at test time by passing corrupted training data points. Pawelczyk et al. [2025] study targeted poisoning (changing some x_{test} 's labels y_{test} to some pre-determined adversarial label), indiscriminate poisoning which aims to reduce the accuracy on the test set overall, and Gaussian poisoning which adds subtle Gaussian noise to training inputs that can later be inferred. On the other hand, Ma et al. [2023] introduce a novel unlearning method inspired by active forgetting in neurology based on neuron masking, coined Forsaken. Ma et al. [2023] find that Forsaken outperforms SISA and full retraining when a proportion of labels of class y are switched to an adversarial another one y_{adv} .

Inspired by both these works, our goal is to study Forsaken's performance across different data poisoning scenarios including Gaussian noise and to provide a deeper empirical understanding of its behavior through targeted diagnostics and hyperparameter analysis.

2 Related Work

- Ma et al. [2023] propose Forsaken, a novel machine unlearning method that leverages neuron masking to actively erase the memorization of targeted training data from neural networks. It introduces a mask gradient generator that produces gradients which, when applied to network neurons, systematically remove the influence of specified samples. A uniform metric called forgetting rate is defined to quantify how effectively the unlearning converts memorized data into non-member status, based on membership inference. Forsaken avoids the heavy overhead of retraining by directly modifying the network's weights without altering its architecture or training procedure. Its design is robust, supporting both out-of-distribution and in-distribution data, and extends even to non-neural models like logistic regression and SVM.
- Thudi et al. [2021] present a comprehensive framework for approximate machine unlearning. It introduces a taxonomy of unlearning metrics and methods, highlighting the "verification error" the L_2 difference between retrained and unlearned model weights as a key metric. They further derive an approximate metric called "unlearning error," which serves as a computationally cheaper proxy for verification error.
- Pawelczyk et al. [2025] examine whether current machine unlearning methods can effectively eliminate the influence of data poisoning attacks. It focuses on three poisoning types: targeted poisoning, indiscriminating poisoning and Gaussian poisoning, where the attacker adds undetectable Gaussian noise solely to the input features, modifying the training inputs to a perturbed version while keeping the labels intact. This last attack scenario constitutes a new diagnosis tool to assess machine unlearning. Experimental results reveal that none of the current unlearning methods consistently remove the effects of these diverse poisoning attacks, underscoring the need for more robust unlearning techniques.

3 Preliminaries

3.1 Forsaken Algorithm

Forsaken is inspired by the theory of active forgetting in neurology where forgetting cells produce a special kind of dopamine that acts as a forgetting signal for engram cells. Those forgetting cells receive feedback from engram cells so they can adjust their dopamine production to avoid excessive forgetting [Ma et al., 2023]. For machine unlearning, forgetting cells are modeled by a mask gradient generator $\mathcal{G}$ which produces mask gradients μ that act like the dopamine signal.

$\mathcal{G}$'s goal is to find a small perturbation over θ_0 that minimize the distance between $\Upsilon = \{y_i | y_i = f_{\theta_0}(x_i), x_i \in D_f\}$, the posteriors generated by the target model f_{θ_0} on the forgetting set D_f and $\mathcal{P}$, the posterior distribution of non-member data. In other words, for an out-of-distribution learning task, the predictions over the forgetting set need to look like an ideal out-of-distribution posterior where entropy is maximal i.e. $f_{\theta} = (y_1, \dots, y_p)$ and $y_1 = \dots = y_p = \frac{1}{p}$. By taking the Kullback-Leibler

divergence as a distance between probability distributions, $\mathcal{G}$ is optimized to minimize $\mathcal{L}_{KL}(\Upsilon, \mathcal{P})$. Ma et al. [2023] add a L_1 regularization term over μ and a dynamic penalty term ω which balances forgetting rates and accuracy drops. ω is defined as $\omega = \frac{1}{D_0} \sum_{x \in D_0} \left| \frac{\partial \mathcal{L}_{crossentropy}(x)}{\partial \theta^{(d)}} \right|$ where $\theta^{(d)}$ is d -th dimension of θ_0 , the parameters of the target model, and D_0 a small proportion (less than 1%) of the training dataset. Finally, the loss used to optimize $\mathcal{G}$ is the following.

$$\mathcal{L}_{forgetting} = \mathcal{L}_{KL}(\Upsilon, \mathcal{P}) + \lambda \omega \|\mu\|_1$$

At this point, $\mathcal{P}$, which is fixed during the whole unlearning process, still needs to be estimated. In order to forget out-of-distribution data, $\mathcal{P}$ needs to estimate the distribution of non-member out-of-distribution data. Such sampling can be done by using irrelevant data to the task at hand. For example, Ma et al. [2023] propose using characters a - z to estimate $\mathcal{P}$ for a digit classification task. Forsaken can be generalized to forgetting in-distribution data by sampling $\mathcal{P}$ over non-member test samples.

Algorithm 1 presents a pseudo-code for Forsaken.

Algorithm 1 Forsaken [Ma et al., 2023]

```

1: procedure FORSAKEN( $f_{\theta_0}, \theta_0, D_f, T, \mathcal{P}$ )
2:   Initialize mask gradient  $\mu_0$ 
3:   for  $t \in \llbracket 1, T \rrbracket$  do
4:      $\mu_t = \mathcal{G}(\theta_{t-1}, \Upsilon)$ 
5:      $\theta_t = \theta_0 - \xi \mu_t$ 
6:      $\Upsilon = \{y_i | y_i = f_{\theta_t}(x_i), x_i \in D_f\}$ 
7:     Optimize  $\mathcal{G}$  by minimizing  $\mathcal{L}_{forgetting}$ 
8:   end for
9:   Return  $f_{\theta_T}$ 
10: end procedure

```

The exact underlying mechanisms behind the mask generator $\mathcal{G}$ remain otherwise somewhat opaque. To our understanding, $\mathcal{G}$ acts as an optimizer over μ :

$$\frac{\partial \mathcal{L}_{forgetting}}{\partial \mu_t} = \frac{\partial \mathcal{L}_{KL}}{\partial \theta_t} \frac{\partial \theta_t}{\partial \mu_t} + \lambda \omega \frac{\partial \|\mu_t\|_1}{\partial \mu_t}$$

Where $\frac{\partial \mathcal{L}_{KL}}{\partial \theta_t}$ and $\frac{\partial \|\mu_t\|_1}{\partial \mu_t}$ are easily computable and $\frac{\partial \theta_t}{\partial \mu_t}$ is obtainable from line 5 of algorithm 1. When "optimizing $\mathcal{G}$ " (line 7 of 1), μ_t is then updated using $\frac{\partial \mathcal{L}_{forgetting}}{\partial \mu_t}$.

3.2 Metrics

Verification Error : The verification error [Thudi et al., 2021] is a central metric for evaluating approximate unlearning methods. It measures the L_2 distance between the weights of the model obtained after an approximate unlearning procedure and those of the ideal model, which is obtained by retraining the model from scratch without the data to be forgotten. Formally, if we denote by w_T the weights of the unlearned model and by w_c the weights of the retrained model, the verification error is defined as

$$\varepsilon_v = \|w_T - w_c\|_2$$

This metric is of great interest because a small verification error indicates that the unlearned model is close to the ideal model, suggesting that the influence of the forgotten data has been effectively mitigated [Thudi et al., 2021]. For instance, if we assume that each weight distribution $P_I(w)$ (corresponding to a given update order during training) is Lipschitz with constant L , and if we define

$$v = \frac{1}{|\mathcal{I}|} \sum_{I \in \mathcal{I}} \|d_I + u_I\|_2,$$

where d_I represents the difference induced by the removal of the data to be forgotten for a given update order I , and u_I represents the update applied by the unlearning method, then one can show that

$$\|P''(w) - P'(w)\|_2 \leq L \cdot v.$$

Here, $P''(w)$ and $P'(w)$ denote the density functions of the weights of the approximatively unlearned model and the retrained model, respectively. This inequality demonstrates that the verification error serves as a proxy for other metrics based on distances in the weight or output space, encompassing a wide range of evaluation criteria for unlearning. In summary, a low verification error indicates that the unlearning method has effectively removed the influence of the forgotten data while maintaining high similarity with the model that would have been obtained through complete retraining.

Unlearning Error : The unlearning error [Thudi et al., 2021] is introduced as a computationally efficient proxy for the verification error. In contrast to the verification error, which requires retraining a model from scratch to obtain an ideal reference, the unlearning error can be estimated directly from the training process by analyzing the influence of the forgotten data on the training updates. By expanding the recursive updates performed by stochastic gradient descent (SGD), we can isolate the terms that depend on the forgotten data. In particular, if we denote by w_t the weights after t updates and by w_0 the initial weights, then the cumulative effect of the training updates can be decomposed into a sum of gradients that are independent of the order and a series of higher-order terms. The dominant contribution from these higher-order terms is proportional to the product of the learning rate squared η^2 , the change in weights $\|w_t - w_0\|_2$, and the average singular value of the Hessian σ_{avg} , scaled by a factor that depends on the number of updates. Formally, we define the unlearning error ε_u as

$$\varepsilon_u = \eta^2 \cdot \frac{\|w_t - w_0\|_2}{t} \cdot \sigma_{\text{avg}} \cdot \frac{t^2 - t}{2}.$$

A lower unlearning error suggests that the model has undergone a smaller effective shift due to the forgotten data, which in turn implies that the approximate unlearning method has better succeeded in erasing the influence of the target samples.

3.3 Poisoning attacks

Data poisoning attacks involve altering subtly a fraction of training data in order to cause a targeted model to behave unpredictably and undesirably.

Label flipping The goal is to make a model misclassify a specific subset of target test data points to a new adversarial label y_{adv} [Pawelczyk et al., 2025]. Tolpegin et al. [2020] describe a simple targeted poisoning attack. Given a source class c_{src} and a target class c_{tgt} , a simple way to make a model misclassify objects of class c_{src} as c_{tgt} is to switch a proportion of the labels of c_{src} training to c_{tgt} . Ma et al. [2023] tested a similar scenario with CIFAR-10 where Forsaken outperformed all methods.

Indiscriminate Data Poisoning. In contrast to targeted or label-specific attacks, *indiscriminate data poisoning* introduces samples from an entirely different distribution into the training set, with the objective of degrading the model’s representation without explicitly altering the test performance under standard metrics. This type of poisoning aims to pollute the learned feature space in subtle ways, making it harder for the model to generalize or to be robust to certain unlearning strategies. The adversary does not aim to manipulate the model into specific behaviors for selected inputs, but rather to inject unrelated patterns into the learned representations.

Gaussian Data Poisoning was introduced by Pawelczyk et al. [2025]. The adversary’s goal is to introduce hidden signals in the training data that do not visibly alter samples but affect model training without hindering model test performance. Targeted datapoints can then easily be inferred from operations on the trained model. Even if poisoned data appears "unlearned" by traditional metrics, Gaussian poisoning reveals whether latent correlations persist in the model’s gradients, and thus represents a diagnostic tool to rigorously evaluate unlearning algorithms. Given a poison budget b , a perturbation ε , $b|D_{\text{train}}|$ $z = (x, y)$ samples are chosen to be poisoned. The poisoned samples are then $x' = x + \xi_z$ where $\xi_z \sim \mathcal{N}(0, \varepsilon^2 I_d)$ if x is of dimension d . After training, the model’s sensitivity to the poisoned data is analyzed using gradients. $g_z = \nabla_x \ell(\theta, (x_{\text{base}}, y))$ is the gradient of the loss function ℓ with respect to the clean input x_{base} . If unlearning fails, the gradients g_z will retain a statistical relationship with the noise ξ_z added during poisoning. To quantify the relationship between gradients and noise, we compute a normalized inner product: $I_z = \frac{\langle g_z, \xi_z \rangle}{\varepsilon_p \|g_z\|_2}$. For a perfectly unlearned model, I_z should follow a standard normal distribution ($\mathcal{N}(0, 1)$), as gradients and noise become statistically independent. The final step aggregates results across all poisoned samples to

compute the **Gaussian Unlearning Score (GUS)**:

$$\text{GUS} = \frac{1}{|S_{\text{poison}}|} \sum_{z \in S_{\text{poison}}} I_z$$

A GUS close to 0 indicates successful unlearning. The model no longer "remembers" the noise pattern. A large absolute GUS ($\gg 0$) implies residual correlation, meaning unlearning failed.

4 Experiments

4.1 Novelty of our work

The original Forsaken paper [Ma et al., 2023] already explores several out-of-distribution unlearning scenarios and briefly touches upon label flipping. While initially designed for and more efficient in those scenarios, Forsaken also outperforms other methods for in-distribution unlearning [Ma et al., 2023]. Our work brings additional experimental depth and diagnostic tools.

We validate Ma et al. [2023] experiments on MNIST and we test Forsaken on Gaussian data poisoning, a subtle form of contamination where traditional accuracy metrics are insufficient to detect memorization. To evaluate unlearning in this setting, we use the Gaussian Unlearning Score (GUS), a gradient-based measure originally proposed in prior work [Pawelczyk et al., 2025] but never applied to Forsaken. We also analyze Forsaken’s behavior on large-scale label flipping (e.g., 50% class switch).

Rather than proposing a new method, our contribution lies in providing a more thorough and diversified evaluation of Forsaken, especially in settings where forgetting is harder to quantify.

4.2 Experimental set up

Experiments were ran using simple fully connected networks with two hidden layers of 256 and 128 neurons respectively, and ReLU activations. MNIST was used as a base for in-distribution unlearning, label poisoning and gaussian poisoning while CIFAR-10 images were added to MNIST to study out-of-distribution learning.

For each type of poisoning, a target model f_{θ_0} was trained on a corrupted dataset $D_{\text{train}} = D_f \sqcup D_c$ where D_f is the forget set (n_f corrupted images for attacks scenarios or clean MNIST images to be forgotten for in-distribution unlearning) and D_c the remaining uncorrupted samples from the original learning set. During f_{θ_0} ’s training, its unlearning error ε_u is computed. A clean model f_{θ_c} was trained on D_c in order to track verification error ε_v throughout the unlearning process of the target model. With Forsaken and T iterations, a third model f_{θ_T} is obtained and serves as a baseline to measure whether unlearning was successful or not. All models were trained with a stochastic gradient descent and a learning rate $\eta = 0.01$. For in-distribution and label poisoning unlearning, f_{θ_0} was trained on MNIST for 5 epochs, for Gaussian poisoning unlearning f_{θ_0} was trained subset of 10 000 data points of the MNIST training set over 25 epochs, for out-of-distribution f_{θ_0} was trained on a subset of 1 200. Indiscriminate poisoning attacks are simulated by injecting a set of images from the CIFAR-10 dataset into the MNIST training set. The CIFAR-10 images are first resized and converted to grayscale to ensure dimensional compatibility with MNIST inputs. Let D_{MNIST} denote the original dataset, and $D_{\text{poison}} \subset \text{CIFAR-10}$ be a set of unrelated but well-formed images. Given a poisoning ratio ρ , we randomly sample $\rho|D_{\text{MNIST}}|$ images from CIFAR-10 and concatenate them to the original MNIST training data, forming the poisoned dataset $D'_{\text{train}} = D_{\text{MNIST}} \cup D_{\text{poison}}$.

In order to assess Forsaken’s effect, accuracy on D_f and the MNIST test dataset D_{test} compared to f_{θ_c} are the primary indicators. The goal is to achieve the same accuracy as a retrained model on the forget set D_f without degrading the accuracy on D_{test} . Additionally, for Gaussian poisoning attacks, the absolute Gaussian Unlearning Score is used to assess whether unlearning is successful, as defined by Pawelczyk et al. [2025]. Since membership inference attacks proved difficult as discussed in Section B, the forgetting rate as defined by Ma et al. [2023] was dropped during unlearning evaluation.

In order to estimate P , $n_{\text{sample}} = 500$ images from the test set are split and used. The model is effectively trained on the remaining samples. Since MNIST is relatively easy to learn, the sampled P is often experimentally close to an ideal out-of-member distribution. Thus, for label poisoning

and out-of-distribution learning, $P(k) = \frac{1}{10}$ for all MNIST classes $k \in \llbracket 0, 9 \rrbracket$, without hindering unlearning performances.

4.3 Forsaken Implementation

Forsaken was implemented in Python and PyTorch from scratch, as no open source code is available at the moment. Our implementation can be viewed on this [GitHub repository](#).

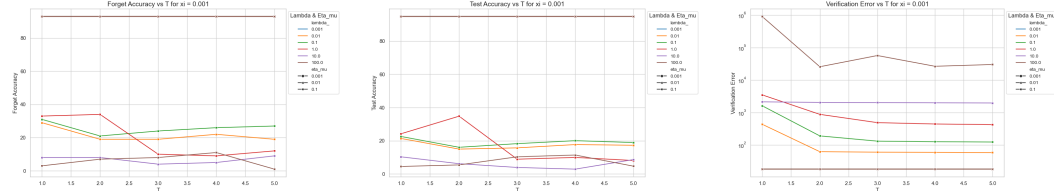
Ma et al. [2023] recommends optimizing the mask gradient μ with L-BFGS, as its convergence is more stable. SGD and L-BFGS were both implemented and use a learning rate $\eta_\mu = 0.01$. With L-BFGS, Ma et al. [2023] recommends using $T \sim 10^1$. Depending on the value of ξ , the magnitude of the parameter update in line 5 of algorithm 1 and the complexity of the noise in D_f the optimal value of T can vary drastically. Indeed, with a large ξ , Forsaken makes the target model f_{θ_0} catastrophically forget its task, even with a small T . With a small enough ξ the effect of unlearning takes many iterations to be seen i.e $T \sim 10^2$. Considering a single iteration, SGD is naturally quicker. However with certain combinations of hyperparameters, $T \sim 10^3$ is required to match L-BFGS performance on 5 iterations. Thus, L-BFGS is preferred throughout the experiments unless otherwise specified.

λ , the fixed penalty term for the L_1 norm of the mask gradient $\|\mu\|_1$ is used to prevent over-forgetting by controlling and prevent divergence of the forgetting signal encoded by μ [Ma et al., 2023]. A random subset of $p\% \leq 1\%$ of the clean training set D_c is used to compute ω . This term is a dynamic penalty term on $\|\mu\|_1$. Computations led to $\omega \sim 10^3$, which caused divergence of loss values $\mathcal{L}_{forgetting}(\Upsilon, P) \approx 10^6$ after around 5 iterations of Forsaken. This is probably due to the combination of a too large ω as compared to λ and $\|\mu\|_1$. As a result, ω is set to $\omega = 1$ unless otherwise stated.

Finally, the most relevant hyperparameters of our study are T , ξ and λ .

4.4 In-distribution unlearning

Forsaken was first tested on in-distribution unlearning tasks, as they are the easiest and fastest to implement. The goal was to gain a sense of basic understanding on Forsaken’s behavior. From MNIST, $n_f = 100$ samples were retrieved to constitute the forget set D_f .



(a) Accuracy on D_f against T for different combinations of (λ, η_μ) . (b) Accuracy on D_{test} against T for different combinations of (λ, η_μ) . (c) ε_v against T for different combinations of (λ, η_μ) .

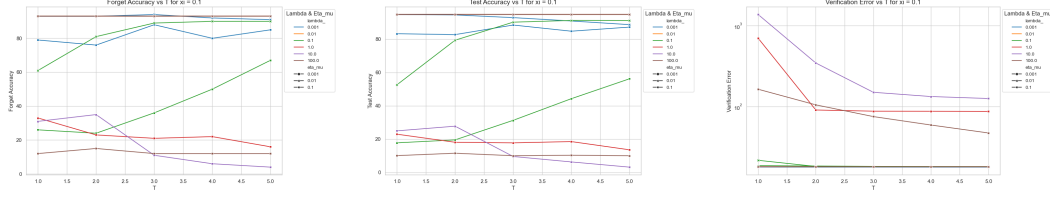
Figure 1: Accuracy on D_f and D_{test} and ε_v against T for $\xi = 0.001$

A hyperparameter search was conducted with a model f_{θ_0} achieving 93% accuracy on D_f and 94.61 % accuracy on D_{test} , those values constitute the plateaus seen in Figures 1 and 2. The f_{θ_T} were obtained with $T \in \llbracket 1, 5 \rrbracket$.

With ξ sufficiently small (Figure 1), Forsaken either unlearns nothing plateauing at the original accuracy on D_f and at $\varepsilon_v = 18.1$ for all iterations or catastrophically unlearns the forgetting set D_f and the test set with diverging ε_v .

Remarkably, lowering verification error ε_v leads to catastrophic unlearning too. As slight drop of 0.1718 from original verification error $\varepsilon_v = 18.1$ leads to accuracy on D_f to drop to 76% and on D_{test} to 82.77 %. This is probably due to the nature of the MNIST dataset. Forgetting a subset of digits here makes the target model forget how to recognize other digits altogether.

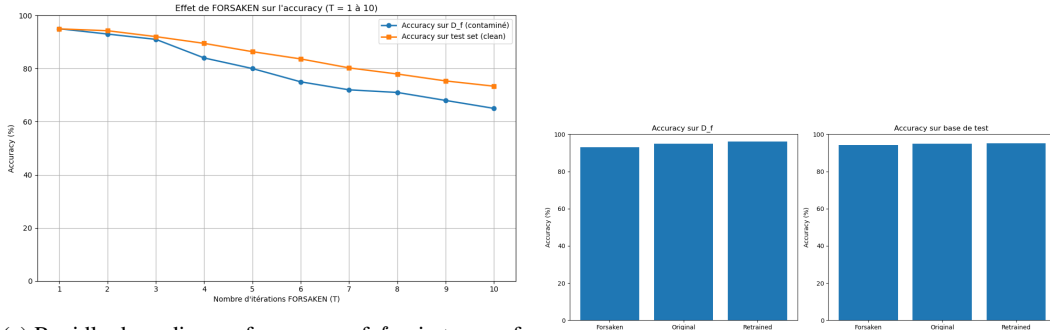
With a larger ξ (Fig. 2), Forsaken is able to obtain better accuracy on D_f and D_{test} . Using an intermediate value of λ , the penalization of $\|\mu\|_1$ effectively allows the model to adapt its forgetting



(a) Accuracy on D_f against T for different combinations of (λ, η_μ) . (b) Accuracy on D_{test} against T for different combinations of (λ, η_μ) . (c) ε_v against T for different combinations of (λ, η_μ) .

Figure 2: Accuracy on D_f and D_{test} and ε_v against T for $\xi = 0.1$

signal and mitigate over-unlearning. This translates to a decreasing ε_v and an increasing accuracy on both D_f and D_{test} for $\lambda = 0.1$ for example.



(a) Rapidly degrading performances of f_{θ_T} in terms of accuracy on test and forget test across iterations.

(b) Performances of f_{θ_2}

Figure 3: Example of in-distribution unlearning.

As an example of effective unlearning, we unlearned f_{θ_0} with $\varepsilon_u(f_{\theta_0}) = 25.33$ with the best set of parameters: $\lambda = 0.001$, $\eta_\mu = 0.01$ and $\xi = 0.1$. The performances of f_{θ_T} against T is represented in Figure 3a. During forgetting, ε_v seems again unable to go below 18. At $T = 2$, f_{θ_2} is able to match f_{θ_c} 's accuracy on D_{test} at 95.17% with 94.25% and D_f at 96% with 93%.

Forsaken is thus effectively capable of unlearning as measured by precision drop without causing significant damage, it is rather controlling the drop that will require care and in-depth tuning in subsequent experiments.

4.5 Gaussian poisoning unlearning

10% of the training set was poisoned using $\varepsilon = 0.05$. The subtle alteration of input causes a drop in accuracy of approximately 1 to 2% on the test set.

As proposed by Pawelczyk et al. [2025], we use the Gaussian unlearning score which quantifies the statistical alignment between model gradients and the injected noise. Ideally, a successfully unlearned model should exhibit a Gaussian unlearning score close to zero, indicating that gradients no longer retain information about the poisoning vector.

A sweep over ξ , $\eta_\mu = 0.01$ over $T = 5$ iterations yielded the best results seen in Figure 4a. Unlearning was based on f_{θ_0} with a base Gaussian unlearning score of $GUS_0 = -0.4280$, an accuracy of 89.77% on the test set and an accuracy of 92.50% on the forget set. Figure 4a effectively highlights a threshold for ξ at which it is big enough to allow unlearning as measured by Gaussian unlearning score without degrading performances significantly.

Figure 5a shows how accuracy on both D_f and the test set evolves as the number of Forsaken iterations increases. As expected, accuracy on D_f drops rapidly, but after a certain point, excessive unlearning also starts to harm test performance. The closest score to 0 is achieved by f_{θ_6} , as per Figure 4b, though not as low as the retrained model's score ($GUS_c = 0.0115$), f_{θ_6} achieves a GUS of $GUS_6 = -0.0198$ with a 86.08% test accuracy and a 73.80% forget accuracy.

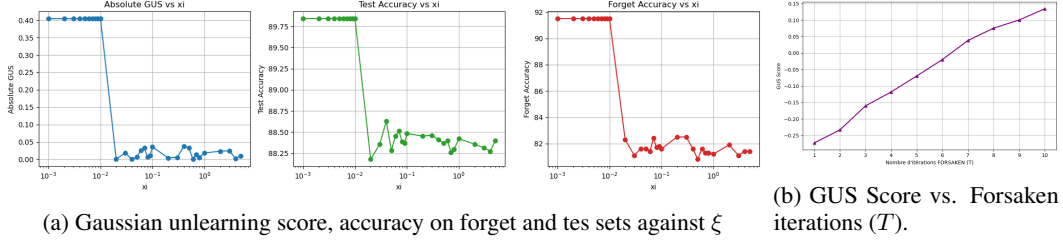


Figure 4: Results on Gaussian unlearning scores. An absolute Gaussian unlearning score close to 0 indicates unlearning, as the model’s gradients are de-correlated from the injected noise.

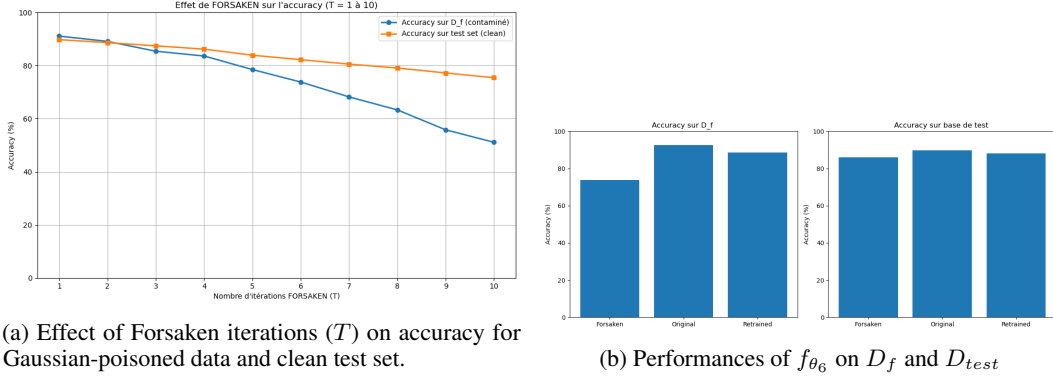


Figure 5: Test and forget accuracy with Gaussian unlearning

Forsaken proves capable of removing Gaussian hidden signals, effectively reaching a GUS close to 0, although leading to a 4% loss in test accuracy.

4.6 Label Poisoning unlearning

In our experiment, we selected class "1" in MNIST and flipped 50% of its labels to "7". Although this 50% poisoning rate might seem aggressive, it was chosen to clearly degrade model performance and test the robustness of unlearning strategies.

Figure 6a compares test accuracy and accuracy on the poisoned set D_f for the three models. Forsaken significantly reduces accuracy on D_f , indicating successful forgetting. Notably, the Forsaken model outperforms the original corrupted model on the test set, showing that forgetting helps mitigate the damage caused by the poisoned labels.

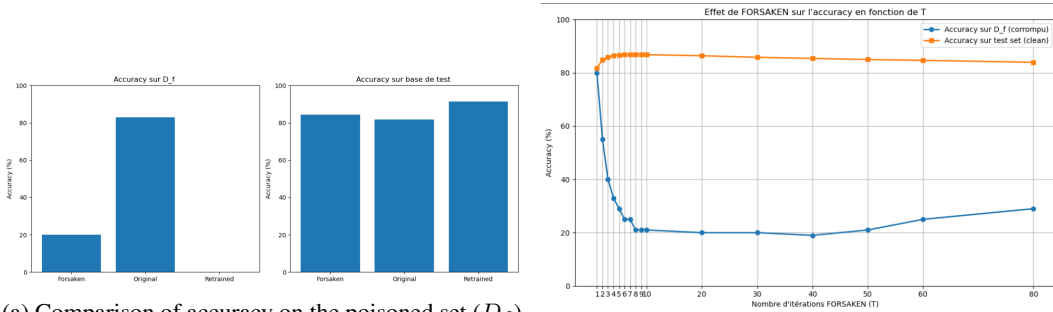


Figure 6: Results of unlearning for label poisoning

Figure 6b shows how the accuracy on D_f and the clean test set evolves across Forsaken iterations (T). We observe that as T increases, accuracy on D_f drops rapidly, while accuracy on the test set increases slightly before stabilizing. This confirms the effectiveness of Forsaken in forgetting targeted poisoned data while preserving and even enhancing the model’s performance on clean data.

4.7 Out-of-Distribution unlearning

To ensure the poisoning had a noticeable impact, we injected 200 or 20% of CIFAR-10 images to a training subset of 1000 MNIST images.

Figure 7a shows that the Forsaken model significantly lowers its accuracy on D_f , indicating successful unlearning. Forsaken maintains performance on the clean test set, with only a slight drop (around 1%) in accuracy compared to the corrupted model. However, as shown in Figure 7b, increasing the number of Forsaken iterations beyond a certain point leads to a noticeable degradation in performance on both the poisoned and clean data. This suggests that the model may begin to over-forget, erasing not only the influence of the targeted data but also useful learned features.

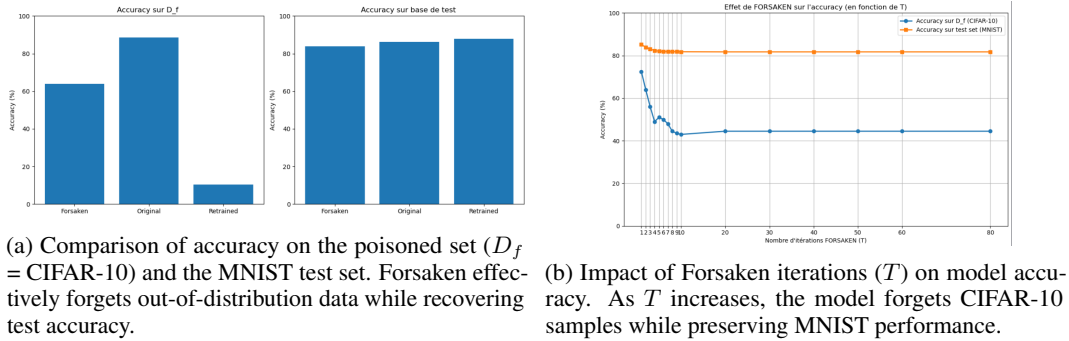


Figure 7: Results of unlearning for out-of-distribution data

5 Conclusion

In this study, we revisited the Forsaken unlearning framework with the goal of providing a more comprehensive empirical analysis across diverse contamination scenarios. We explored label poisoning, Gaussian data poisoning, and out-of-distribution contamination.

Our results confirm that Forsaken can effectively reduce memorization of poisoned data. In particular, for aggressive label poisoning scenarios, we observed that Forsaken not only forgets the corrupted subset, and in some cases even outperforms the original corrupted model on clean test data. These findings highlight Forsaken’s ability to mitigate harmful learning effects when properly tuned.

To that end, we conducted hyperparameter searches to understand the impact of key parameters (penalty term λ , forgetting rate ξ and number of iterations T) and identified the balance needed between forgetting strength and generalization. Additionally, we used the Gaussian Unlearning Score (GUS) as a gradient-based diagnostic tool to assess unlearning effectiveness in Gaussian poisoning scenarios where test accuracy alone fails to reflect residual memorization.

Overall, our work complements the original Forsaken paper by extending its evaluation to new poisoning strategies such as Gaussian noise and large-scale label flipping, and by conducting a hyperparameter study to better apply the unlearning.

References

- L. Bourtoutle, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot. Machine unlearning. *CoRR*, abs/1912.03817, 2019. URL <http://arxiv.org/abs/1912.03817>.
- Y. Cao and J. Yang. Towards Making Systems Forget with Machine Unlearning. In *2015 IEEE Symposium on Security and Privacy (S&P)*, pages 463–480, 2015. doi: 10.1109/SP.2015.35.
- Z. Ma, Y. Liu, X. Liu, J. Liu, J. Ma, and K. Ren. Learn to Forget: Machine Unlearning via Neuron Masking. *IEEE Transactions on Dependable and Secure Computing*, 20(4):3194–3207, 2023. doi: 10.1109/TDSC.2022.3194884.
- T. T. Nguyen, T. T. Huynh, Z. Ren, P. L. Nguyen, A. W.-C. Liew, H. Yin, and Q. V. H. Nguyen. A Survey of Machine Unlearning. *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2209.02299>.
- M. Pawelczyk, J. Z. Di, Y. Lu, G. Kamath, A. Sekhari, and S. Neel. Machine Unlearning Fails to Remove Data Poisoning Attacks. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=HaX48yksVL>.
- A. Thudi, G. Deza, V. Chandrasekaran, and N. Papernot. Unrolling SGD: Understanding Factors Influencing Machine Unlearning. *CoRR*, abs/2109.13398, 2021. URL <https://arxiv.org/abs/2109.13398>.
- V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu. Data poisoning attacks against federated learning systems. *CoRR*, abs/2007.08432, 2020. URL <https://arxiv.org/abs/2007.08432>.

A Forgetting rate

A.1 Forgetting Rate

Forgetting rate [Ma et al., 2023] : This metric is based on the concept of membership inference, which assesses whether a given data sample is recognized as having been part of the original training set. Formally, let B_T denote the number of samples in the forget set that were originally identified as members by a membership oracle, and let B_F represent the number of those samples that were predicted as non-members before the unlearning process. After unlearning is performed, let A_F be the number of samples that are now predicted as non-members. The forgetting rate (FR) is then defined as

$$\text{FR} = \frac{A_F - B_F}{B_T}.$$

A forgetting rate of 100% indicates that all the samples originally recognized as members have been successfully unlearned (i.e., the model now treats them as non-members), whereas a forgetting rate of 0% implies that the unlearning method has had no effect. This metric provides an intuitive measure of the transformation of the forget set from “memorized” to “unknown” status, and it serves as a key indicator for evaluating how well the unlearning algorithm erases the influence of the targeted data.

B Membership Inference Attacks on MNIST

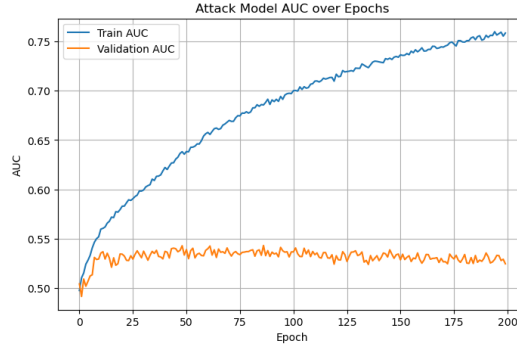


Figure 8: Train and validation area under the receiver operation characteristic curve (ROC AUC score). A score around 0.5 means the model is close to random guessing, while a score of 1 means the model is correctly classifying points.

A single shadow model with the same architecture as the target model was trained on half of MNIST. The shadow model overfit with an accuracy of 100% on its training set and an accuracy of 96% on its test set, so as to distinguish member data from non-member data. Extracted features feeding the binary classifier attack model were composed of output logits, confidence and entropy of the predictions of the shadow model. The attack model is a simple neural net with two hidden layers of size 128 and 64. While it achieved an Area Under the Receiver Operating Characteristic Curve (ROC AUC) of around 0.7 on its training set, the ROC AUC score converges near 0.53 on the validation set as seen in Figure 8. This indicates the attack model is close to random guessing and cannot be used reliably to perform member inference attacks or compute the forgetting rate. Using another type of classifier, like an XGBoost classifier, as mentioned by Ma et al. [2023] could help remedy the issue.